

Jesús Miguel

González Marca

22200019

20/06/26

Sección 1

- 1.1 El repositorio es la 'única fuente de Verdad' porque centraliza toda la lógica de acceso a datos, la UI solo interactúa con el repositorio sin conocer si los datos provienen de Room (local) o Retrofit (remoto), y actúa como fachada porque oculta esa complejidad. Su flujo es que primero devuelve datos cacheados de Room y si están desactualizados llama a la API, guarda los datos nuevos en Room y los envía a la UI via Flow.
- 1.2 ViewModelScope es una CoroutineScope ligado al ciclo de vida del ViewModel, es importante porque cuando el usuario cierra la pantalla, el ViewModel se destruye y el scope cancela de forma automática todas las corrutinas activas liberando memoria y evitando memory leaks, una petición de red iniciada en ese scope se cancela antes de completarse.
- 1.3 Un flow (frío) no envía datos hasta que hay un recolector, un Stateflow (caliente) existe independientemente de los colectores, siempre tiene un valor actual y los nuevos colectores reciben inmediatamente el último estado, en UI se prefiere Stateflow porque garantiza que la pantalla siempre tenga un estado válido al recomponerse.
- 1.4 Se crea una clase que extiende Application y se declara en el AndroidManifest, dentro se instancian los objetos de infraestructura usando lazy lo cual significa que la instancia solo se crea la primera vez que se accede, no al arrancar la app, todos los componentes (Actividad, ViewModel) acceden a ellos mediante (Application as Miapp).repository, lo cual garantiza una única instancia compartida.

Gonzales Marca Jesus Miguel

Seccion 2

- 2.1 El flujo de datos que emite automáticamente cada vez que una tabla de Room cambia se implementa usando el tipo Flow.
- 2.2 El operador Dispatchers.Default permite ejecutar tareas pesadas fuera del Main Thread, específicamente optimizado para tareas del CPU como parseo masivo de datos.
- 2.3 Para que un servicio en primer plano de ubicación sea válido en Android 10+, debe declararse en el manifiesto con el tipo location.
- 2.4 La función getInstance() de la clase Application es la que garantiza que los componentes compartan la misma instancia del repositorio mediante un 'cast'.
- 2.5 El componente que permite transformar una API basada en callbacks antiguos en una función suspendible moderna se llama SuspendCancellableCoroutine.
- 2.6 El archivo de metadatos que describe la estructura de la app al sistema operativo se llama exactamente AndroidManifest.xml.
- 2.7 En la arquitectura AOSP, la capa que actúa como puente estándar entre el framework y los controladores se denomina HAL (hardware abstraction layer).
- 2.8 El operador de Flow que permite combinar múltiples puentes (como GPS y sensores) para emitir un estado unificado es combine.

Sección 3 Código Kotlin con PascalCase para clases y camelCase para variables

3.1 Repositorio y DI

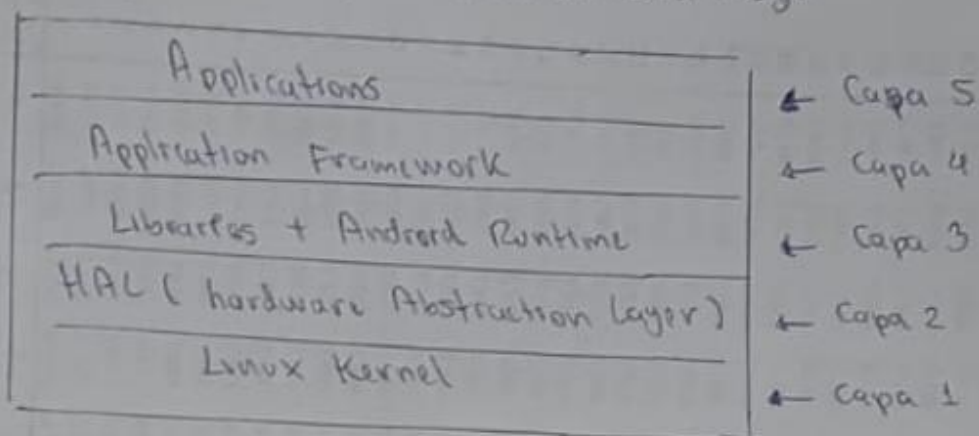
```
class UsuarioRepository (private val dao: UsuarioDao) {  
    suspend fun obtenerUsuarios () = dao.getAll()  
}  
  
class MiApp : Application () {  
    val database : AppDatabase by lazy {  
        Room.databaseBuilder (this, AppDatabase::class.java,  
            "app-db").build ()  
    }  
    val repository : UsuarioRepository by lazy {  
        UsuarioRepository (database.usuarioDao ())  
    }  
}
```

3.2 Lógica de identidad

```
data class Usuario (  
    val nombre : String,  
    val apellidos : String,  
    val edad : Int?  
)  
  
fun main () {  
    val usuario = Usuario (  
        nombre = "Jesus",  
        apellidos = "Gonzales Marco",  
        edad = 25  
    )  
    val puntaje = (usuario.nombre.replace (" ", "").length  
        + usuario.apellidos.replace (" ", "").length)  
        ?: 0 // operador Elvis por si fueran nulos  
    println ("Usuario: { ${usuario.nombre} } { ${usuario.apellidos} }," +  
        "tu puntaje de identidad es { $puntaje }")  
}
```

Sección 4

4.1 Diagrama 5 capas de arriba hacia abajo



4.2 Descripción de capas

Applications : Su función es contener las apps del usuario y del Sistema, como gmail, ajustes, etc.

Application Framework : APIs Java para desarrolladores: Activity Manager, Content Providers, Window Manager

Libraries + Android Runtime : Contiene las librerías nativas (SQLite, OpenGL) + ART (el cual se encarga de ejecutar el código de las apps)

HAL : Interfaz estándar entre el framework y los drivers del hardware

Linux Kernel : Se encarga de la gestión de memoria, procesos, drivers, seguridad.

Sobre el ART (android runtime) contiene: el compilador AOT (Ahead-of-Time), el compilador JIT, el Garbage Collector y las Core Libraries de Java/Kotlin